

```

"""
Filament_absorber_CF_profile.py

Calculate the Ly-alpha covering fraction as a function of
filament-centric distance and compare it with the fitted
exponential covering-fraction model and the expected IGM
contribution.
"""

# =====
# Imports
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from astropy.stats import binom_conf_interval
from matplotlib.ticker import MultipleLocator

from IGM_CF import IGM_CF

# =====
# Configuration
# =====

INPUT_FILE = "Filament_sample_318.csv"

# =====
# Read data
# =====

data = pd.read_csv(INPUT_FILE)
rb_data_all = data.copy()

# =====
# Select data for covering-fraction analysis
# =====

ew_th = float(
    input("Please Enter your EW threshold value: ")
)

condition = (
    rb_data_all["ew_noise_lya"] <= ew_th
)

rb_new_data = rb_data_all.loc[
    condition
].copy()

# =====
# Distance bins
# =====

bins = np.array([
    0, 1, 2, 4, 8, 26
])

centres = 0.5 * (
    bins[:-1] + bins[1:]
)

lows = bins[:-1]
highs = bins[1:]

# =====
# Initialize covering-fraction arrays
# =====

CF_RK = []

Bins_RK = []
Bins_med_low_err = []
Bins_med_high_err = []

N = []
N_total = []

CF_low_err_RK = []
CF_high_err_RK = []

confidence_level = 0.6826
method = "wilson"

# =====
# Calculate covering fraction
# =====

for i in range(len(centres)):

    lo = lows[i]
    hi = highs[i]

    # Include the upper boundary only for the final bin.
    if i < len(centres) - 1:
        mask = (
            (rb_new_data["D_Fil"] >= lo)
            & (rb_new_data["D_Fil"] < hi)
        )
    else:
        mask = (
            (rb_new_data["D_Fil"] >= lo)
            & (rb_new_data["D_Fil"] <= hi)
        )

    new_table = rb_new_data.loc[mask]

    # -----
    # Extract quantities
    # -----

    pair_id = np.array(
        new_table["pair_id"]
    )

    CF = np.array(

```

```

        new_table["flag"]
    )

ew = np.array(
    new_table["EW_total"]
)

ew_up = np.array(
    new_table["upper_limit"]
)

D_Fil = np.array(
    new_table["D_Fil"]
)

# -----
# Median distance and 68% interval
# -----

if D_Fil.size > 0:

    median_dm = np.median(D_Fil)

    p16 = np.percentile(
        D_Fil,
        16
    )

    p84 = np.percentile(
        D_Fil,
        84
    )

    med_err_low = (
        median_dm - p16
    )

    med_err_high = (
        p84 - median_dm
    )

else:

    median_dm = np.nan
    med_err_low = np.nan
    med_err_high = np.nan

Bins_RK.append(median_dm)

Bins_med_low_err.append(
    med_err_low
)

Bins_med_high_err.append(
    med_err_high
)

# -----
# Ly-alpha detections
# -----

num_condition = (
    (CF >= 3)
    & (ew > ew_th)
)

num = int(
    len(ew[num_condition])
)

den = int(
    len(np.unique(pair_id))
)

N.append(num)
N_total.append(den)

# -----
# Covering fraction and Wilson confidence interval
# -----

if den > 0:

    phat = num / den

    CF_RK.append(phat)

    low_ci, high_ci = binom_conf_interval(
        num,
        den,
        confidence_level=confidence_level,
        interval=method
    )

    lower = float(
        np.clip(low_ci, 0.0, 1.0)
    )

    upper = float(
        np.clip(high_ci, 0.0, 1.0)
    )

    err_low = abs(
        phat - lower
    )

    err_up = abs(
        upper - phat
    )

    CF_low_err_RK.append(
        err_low
    )

    CF_high_err_RK.append(
        err_up
    )

else:

    CF_RK.append(np.nan)

    CF_low_err_RK.append(

```

```

        np.nan
    )

    CF_high_err_RK.append(
        np.nan
    )

# =====
# Convert to arrays
# =====

CF_RK = np.asarray(
    CF_RK,
    dtype=float
)

Bins_RK = np.asarray(
    Bins_RK,
    dtype=float
)

Bins_med_low_err = np.asarray(
    Bins_med_low_err,
    dtype=float
)

Bins_med_high_err = np.asarray(
    Bins_med_high_err,
    dtype=float
)

N = np.asarray(
    N,
    dtype=int
)

N_total = np.asarray(
    N_total,
    dtype=int
)

CF_low_err_RK = np.asarray(
    CF_low_err_RK,
    dtype=float
)

CF_high_err_RK = np.asarray(
    CF_high_err_RK,
    dtype=float
)

# =====
# Covering-fraction results
# =====

cf_p3 = CF_RK

bins_p3 = Bins_RK

cf_p3_err = np.vstack([
    CF_low_err_RK,
    CF_high_err_RK
])

bins_p3_err = np.vstack([
    Bins_med_low_err,
    Bins_med_high_err
])

# =====
# Covering-fraction Exponential Model results
# =====

CF_model = pd.read_csv("./exp_fit_outputs/exp_fit_curve.csv")

# =====
# IGM contribution
# =====

igm_cf_p3_low, igm_cf_err_p3_low = IGM_CF(
    ew_th,
    1215.67,
    0.416,
    2400,
    min(rb_new_data["z_cls"])
)

igm_cf_p3, igm_cf_err_p3 = IGM_CF(
    ew_th,
    1215.67,
    0.416,
    2400,
    np.median(rb_new_data["z_cls"])
)

igm_cf_p3_up, igm_cf_err_p3_up = IGM_CF(
    ew_th,
    1215.67,
    0.416,
    2400,
    np.max(rb_new_data["z_cls"])
)

# =====
# Plot
# =====

plt.rcParams["font.family"] = "DejaVu Sans"

fig, ax = plt.subplots(
    figsize=(8.6, 6.8)
)

# -----
# Observed covering fraction
# -----

ax.errorbar(

```

```

bins_p3,
cf_p3,
xerr=bins_p3_err,
yerr=cf_p3_err,
marker="s",
markersize=20,
markeredgecolor="k",
color="b",
linewidth=0,
markeredgewidth=2,
elinewidth=3,
capsize=4,
alpha=0.8,
zorder=3
)

# -----
# Exponential model
# -----

ax.plot(
    CF_model["D"] / 1000,      # D in kpc
    CF_model["p_median"],
    linewidth=4,
    label=r"Model:  $k_0 + k_1 e^{-D/D_0}$ "
)

ax.fill_between(
    CF_model["D"] / 1000,
    CF_model["p_lo_68"],
    CF_model["p_hi_68"],
    alpha=0.35,
    label="68% Confidence"
)

# -----
# IGM contribution
# -----

ax.axhline(
    y=igm_cf_p3,
    color="k",
    linestyle="--",
    linewidth=3,
    label="IGM",
    alpha=0.8
)

x_min, x_max = plt.xlim(
    0.05,
    30
)

ax.fill_between(
    [x_min, x_max],
    igm_cf_p3_low,
    igm_cf_p3_up,
    color="gray",
    alpha=0.4
)

# -----
# Axes and ticks
# -----

ax.tick_params(
    axis="both",
    length=3,
    width=3,
    which="minor",
    direction="out",
    labelsize=20,
    color="k"
)

ax.tick_params(
    axis="both",
    length=6,
    width=4,
    which="major",
    direction="out",
    labelsize=20,
    color="k"
)

ax.yaxis.set_ticks_position("both")
ax.xaxis.set_ticks_position("both")

ax.minorticks_on()

ax.yaxis.set_major_locator(
    MultipleLocator(0.2)
)

# -----
# Annotation
# -----

ax.text(
    0.12,
    0.70,
    r" $D_0 = 3.3 \sim 5 \text{ pMpc}$ ",
    fontsize=19
)

# -----
# Labels and limits
# -----

ax.set_xscale("log")

ax.set_xlabel(
    r" $D_{\rm Fil}$  [pMpc]",
    fontsize=25
)

ax.set_ylabel(
    r" $f_c \sim (\alpha) \text{ Ly}$ ",
    fontsize=22
)

```

```
)  
  
ax.set_xlim(  
    0.08,  
    30  
)  
  
ax.set_ylim(  
    0.15,  
    0.8  
)  
  
# -----  
# Legend  
# -----  
  
ax.legend(  
    ncol=1,  
    fontsize=18,  
    loc="upper left",  
    labelspacing=1.2,  
    bbox_to_anchor=(0.51, 0.95),  
    frameon=False  
)  
  
plt.tight_layout()  
plt.show()
```

```

"""
Filament_absorber_correlation_CF_profile.py

Calculate the Ly-alpha covering-fraction profile around
cosmic-web filaments and compare it with the prediction
from the filament-absorber clustering model.

The observed covering fraction is calculated in bins of
filament-centric distance. The clustering-model posterior
samples of r0 and gamma are then propagated to obtain the
model covering-fraction profile and its 68% confidence band.

Inputs
-----
Filament_sample_318.csv
input_filament_Clustering_file.csv
./dump_data_clustering/chains/equal_weighted_post.txt

Required columns
-----
Filament_sample_318.csv:
    ew_noise_lya
    z_cls
    D_Fil
    pair_id
    flag
    EW_total

input_filament_clustering_file.csv:
    D_Fil
    z_cls
    det_flag

Output
-----
Observed Ly-alpha covering fraction
IGM covering fraction
Clustering-model covering fraction
68% confidence interval
"""

# =====
# Imports
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import scipy.integrate

from astropy import units as u
from astropy.cosmology import FlatLambdaCDM
from astropy.stats import binom_conf_interval
from scipy.ndimage import gaussian_filter1d

from IGM_CF import IGM_CF

cosmo = FlatLambdaCDM(
    H0=67.4,
    Om0=0.315
)

# =====
# Configuration
# =====

FILAMENT_DATA_FILE = "Filament_sample_318.csv"
CLUSTERING_DATA_FILE = "input_filament_clustering_file.csv"
POSTERIOR_FILE = (
    "./dump_data_clustering/chains/equal_weighted_post.txt"
)

EW_THRESHOLD = 0.3          # Angstrom
DV = 1200.0 * u.km / u.s
NOM = 33.35                 # dN/dz of random absorbers

N_POSTERIOR_DRAWS = 2000
SMOOTHING_SIGMA = 2

# =====
# 1. Observed covering fraction
# =====

data = pd.read_csv(FILAMENT_DATA_FILE)

# Select systems satisfying the EW-noise criterion.
data = data[
    data["ew_noise_lya"] <= EW_THRESHOLD
].copy()

# Convert filament-centric distance from pMpc to cMpc.
# a = 1 / (1 + z), therefore D_comoving = D_physical / a.
a = 1.0 / (1.0 + data["z_cls"])

data["D_Fil"] = data["D_Fil"] / a # Comoving

# =====
# Distance bins
# =====

bins = np.array([0, 1, 3, 6, 12, 24])

centres = 0.5 * (bins[:-1] + bins[1:])
lows = bins[:-1]
highs = bins[1:]

# =====
# Covering-fraction arrays
# =====

CF = []
DISTANCE_MEDIAN = []
DISTANCE_LOW_ERR = []
DISTANCE_HIGH_ERR = []

N_DETECTION = []
N_TOTAL = []

CF_LOW_ERR = []

```

```

CF_HIGH_ERR = []

CONFIDENCE_LEVEL = 0.6826

# =====
# Calculate observed covering fraction
# =====

for i in range(len(centres)):

    lower = lows[i]
    upper = highs[i]

    # Include the upper boundary only in the final bin.
    if i < len(centres) - 1:
        mask = (
            (data["D_Fil"] >= lower)
            & (data["D_Fil"] < upper)
        )
    else:
        mask = (
            (data["D_Fil"] >= lower)
            & (data["D_Fil"] <= upper)
        )

    bin_data = data.loc[mask]

    distances = bin_data["D_Fil"].to_numpy()

    # -----
    # Median distance and 68% interval
    # -----

    if len(distances) > 0:

        median_distance = np.median(distances)

        p16 = np.percentile(distances, 16)
        p84 = np.percentile(distances, 84)

        low_error = median_distance - p16
        high_error = p84 - median_distance

    else:

        median_distance = np.nan
        low_error = np.nan
        high_error = np.nan

    DISTANCE_MEDIAN.append(median_distance)
    DISTANCE_LOW_ERR.append(low_error)
    DISTANCE_HIGH_ERR.append(high_error)

    # -----
    # Ly-alpha detections
    # -----

    detection = (
        (bin_data["flag"] >= 3)
        & (bin_data["EW_total"] > EW_THRESHOLD)
    )

    numerator = int(detection.sum())

    # pair_id is unique for each quasar-cluster pair.
    denominator = int(
        bin_data["pair_id"].nunique()
    )

    N_DETECTION.append(numerator)
    N_TOTAL.append(denominator)

    # -----
    # Covering fraction
    # -----

    if denominator > 0:

        covering_fraction = (
            numerator / denominator
        )

        lower_ci, upper_ci = binom_conf_interval(
            numerator,
            denominator,
            confidence_level=CONFIDENCE_LEVEL,
            interval="wilson"
        )

        lower_ci = float(
            np.clip(lower_ci, 0.0, 1.0)
        )

        upper_ci = float(
            np.clip(upper_ci, 0.0, 1.0)
        )

        CF.append(covering_fraction)

        CF_LOW_ERR.append(
            covering_fraction - lower_ci
        )

        CF_HIGH_ERR.append(
            upper_ci - covering_fraction
        )

    else:

        CF.append(np.nan)
        CF_LOW_ERR.append(np.nan)
        CF_HIGH_ERR.append(np.nan)

# Convert to arrays
CF = np.asarray(CF, dtype=float)

DISTANCE_MEDIAN = np.asarray(
    DISTANCE_MEDIAN,
    dtype=float
)

```

```

DISTANCE_LOW_ERR = np.asarray(
    DISTANCE_LOW_ERR,
    dtype=float
)

DISTANCE_HIGH_ERR = np.asarray(
    DISTANCE_HIGH_ERR,
    dtype=float
)

CF_LOW_ERR = np.asarray(
    CF_LOW_ERR,
    dtype=float
)

CF_HIGH_ERR = np.asarray(
    CF_HIGH_ERR,
    dtype=float
)

N_DETECTION = np.asarray(
    N_DETECTION,
    dtype=int
)

N_TOTAL = np.asarray(
    N_TOTAL,
    dtype=int
)

# =====
# 2. IGM contribution
# =====

igm_cf_low, _ = IGM_CF(
    EW_THRESHOLD,
    1215.67,
    0.416,
    2400,
    data["z_cls"].min()
)

igm_cf, _ = IGM_CF(
    EW_THRESHOLD,
    1215.67,
    0.416,
    2400,
    data["z_cls"].median()
)

igm_cf_high, _ = IGM_CF(
    EW_THRESHOLD,
    1215.67,
    0.416,
    2400,
    data["z_cls"].max()
)

# =====
# 3. Load clustering-model input
# =====

clustering_data = pd.read_csv(
    CLUSTERING_DATA_FILE
)

b_exact = clustering_data["D_Fil"].to_numpy(
    dtype=float
)

z_exact = clustering_data["z_cls"].to_numpy(
    dtype=float
)

obj = clustering_data["det_flag"].to_numpy(
    dtype=int
)

z_med = np.median(z_exact)

# =====
# 4. Convert impact parameter to comoving kpc
# =====

# Input D_Fil is in proper kpc.
# Convert to comoving kpc:
#
#     b_comoving = b_physical * (1 + z)
#
b_comoving = (
    b_exact * (1.0 + z_exact)
)

# =====
# 5. Distance grid
# =====

b_grid = np.logspace(
    np.log10(b_comoving.min()),
    np.log10(b_comoving.max()),
    120
)

# =====
# 6. Load clustering posterior
# =====

samples = np.loadtxt(
    POSTERIOR_FILE,
    skiprows=1
)

r0_samples = samples[:, 0]
gamma_samples = samples[:, 1]

# =====
# 7. Model constants

```

```

# =====

a = 1/(1.0 + z_med)

# =====
# 8. Clustering model
# =====

def compute_model_vals(
    b_array,
    z_med,
    r0,
    gamma,
    nom,
    dv,
    cosmo
):
    """
    Calculate the logarithm of the probability of finding
    zero absorbers at each impact parameter.
    """

    n_points = len(b_array)

    model_values = np.zeros(n_points)

    dz = (
        2.0
        * (dv.value / 3.0e5)
        * (1.0 + z_med)
    )

    n_abs = nom * dz

    H_z = cosmo.H(z_med)

    c1 = (
        a * H_z / (2.0 * dv)
    ).to(
        1 / u.Mpc
    ).value

    lim = (
        dv / (a * H_z)
    ).to(
        u.Mpc
    ).value

    for i in range(n_points):

        def func(r, r0, gamma, b):

            separation = np.sqrt(
                b**2 + r**2
            )

            return (
                c1
                * (separation / r0) ** (-gamma)
            )

        # Convert comoving kpc to comoving Mpc.
        b_mpc = b_array[i] / 1000.0

        integral = scipy.integrate.quad(
            func,
            -lim,
            lim,
            args=(r0, gamma, b_mpc)
        )[0]

        model_values[i] = (
            -(1.0 + integral)
            * n_abs
        )

    return model_values

# =====
# 9. Propagate posterior uncertainties
# =====

rng = np.random.default_rng(
    42
)

indices = rng.integers(
    0,
    len(r0_samples),
    size=N_POSTERIOR_DRAWS
)

r0_draws = r0_samples[indices]
gamma_draws = gamma_samples[indices]

CF_all = []

for r0, gamma in zip(
    r0_draws,
    gamma_draws
):

    model_values = compute_model_vals(
        b_grid,
        z_med,
        r0,
        gamma,
        NOM,
        DV,
        cosmo
    )

    covering_fraction = (
        1.0 - np.exp(model_values)
    )

    CF_all.append(
        covering_fraction
    )

```

```

CF_all = np.asarray(
    CF_all
)

# =====
# 10. 68% confidence interval
# =====

CF_68_low = np.percentile(
    CF_all,
    16,
    axis=0
)

CF_68_median = np.percentile(
    CF_all,
    50,
    axis=0
)

CF_68_high = np.percentile(
    CF_all,
    84,
    axis=0
)

# =====
# 11. Smooth model profile
# =====

CF_low_smooth = gaussian_filter1d(
    CF_68_low,
    SMOOTHING_SIGMA
)

CF_median_smooth = gaussian_filter1d(
    CF_68_median,
    SMOOTHING_SIGMA
)

CF_high_smooth = gaussian_filter1d(
    CF_68_high,
    SMOOTHING_SIGMA
)

# =====
# 12. Plot
# =====

plt.rcParams["mathtext.default"] = "regular"
plt.rcParams["font.family"] = "DejaVu Sans"

fig, ax = plt.subplots(
    figsize=(8, 6)
)

# Observed covering fraction
ax.errorbar(
    DISTANCE_MEDIAN * 1000,
    CF,
    xerr=np.vstack([
        DISTANCE_LOW_ERR * 1000,
        DISTANCE_HIGH_ERR * 1000
    ]),
    yerr=np.vstack([
        CF_LOW_ERR,
        CF_HIGH_ERR
    ]),
    marker="s",
    markersize=10,
    markeredgecolor="black",
    color="blue",
    linewidth=0,
    markeredgewidth=2.0,
    elinewidth=2,
    capsize=4,
    alpha=0.95
)

# Clustering-model profile
ax.plot(
    b_grid,
    CF_median_smooth,
    linewidth=3,
    color="blue"
)

ax.fill_between(
    b_grid,
    CF_low_smooth,
    CF_high_smooth,
    alpha=0.23,
    color="dodgerblue",
    label="68% Confidence"
)

# IGM contribution
ax.axhline(
    igm_cf,
    color="black",
    linestyle="--",
    linewidth=2,
    label="IGM",
    alpha=0.8
)

x_min, x_max = ax.get_xlim()

ax.fill_between(
    [x_min, x_max],
    igm_cf_low,
    igm_cf_high,
    color="gray",
    alpha=0.3
)

```

```

# Annotations
ax.text(
    2500,
    0.75,
    r"$\Delta v = \pm 1200 \sim \rm km \sim s^{-1}$",
    fontsize=17
)

ax.text(
    2500,
    0.65,
    r"$W_{\rm th} = 0.3 \sim \AA$",
    fontsize=17
)

# Axes
ax.set_xscale("log")

ax.set_xlabel(
    r"$D_{\rm Fil}$ (pMpc)",
    fontsize=25
)

ax.set_ylabel(
    r"$f_{\rm c}({\rm Ly})\alpha$",
    fontsize=22
)

ax.set_xlim(
    100,
    27000
)

ax.set_ylim(
    0.15,
    1.05
)

ax.tick_params(
    axis="both",
    which="major",
    length=6,
    width=4,
    direction="out",
    labelsize=20
)

ax.tick_params(
    axis="both",
    which="minor",
    length=3,
    width=3,
    direction="out",
    labelsize=20
)

ax.yaxis.set_ticks_position("both")
ax.xaxis.set_ticks_position("both")

ax.minorticks_on()

ax.legend(
    fontsize=16,
    loc="upper left",
    bbox_to_anchor=(0.55, 1.0),
    frameon=False
)

plt.tight_layout()
plt.show()

```

```

"""
EW_obs_vs_sim_code.py

Model the Ly-alpha equivalent-width profile around cosmic-web
filaments using the filament gas-density model.

The model is evaluated for three filament configurations:

    1. Short filaments
    2. Long filaments
    3. Modified filament model

Uncertainties in the model parameters are propagated using
Monte Carlo sampling.

The resulting EW profiles are compared with the observed
Ly-alpha absorption measurements.

Input
-----
Filament_sample_318.csv

The model parameters are specified below for the short,
long, and modified filament cases.
"""

# =====
# Imports
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# =====
# Load data
# =====

data = pd.read_csv(
    "Filament_sample_318.csv"
)

rb_data_all = data.copy()

# =====
# Prepare EW data
# =====

# Detection flag used for the EW plot.
rb_data_all["det_flag"] = (
    rb_data_all["EW_total"] != -9999
).astype(int)

# Use the measured EW for detections and the EW-noise
# value for non-detections.
rb_data_all["ew"] = np.where(
    rb_data_all["flag"] == 0,
    rb_data_all["ew_noise_lya"],
    rb_data_all["EW_total"]
)

# =====
# Model functions
# =====

def get_stats(arr):
    """
    Calculate the median and 16th/84th percentiles.
    """

    return (
        np.median(arr, axis=0),
        np.percentile(arr, 16, axis=0),
        np.percentile(arr, 84, axis=0)
    )

def run_filament_model(
    r,
    params,
    n_samples=5000,
    seed=None
):
    """
    Calculate the filament density, temperature, H I column
    density, and Ly-alpha EW profile using Monte Carlo
    sampling of the model parameters.

    Parameters
    -----
    r : ndarray
        Filament-centric distance.

    params : dict
        Model parameters and their uncertainties.

    n_samples : int
        Number of Monte Carlo realizations.

    seed : int or None
        Random seed.

    Returns
    -----
    dict
        Median, 16th percentile, and 84th percentile profiles
        for delta, logT, NHI, and EW.
    """

    if seed is not None:
        np.random.seed(seed)

    # -----
    # Nominal model parameters
    # -----

    delta_0 = params["delta_0"]
    alpha = params["alpha"]
    beta = params["beta"]

```

```

r0 = params["r0"]

a = params["a"]
b = params["b"]

rho_avg = params["rho_avg"]
rho_crit = params["rho_crit"]

Gamma_12 = params["Gamma_12"]
z = params["z"]

# -----
# Parameter uncertainties
# -----

delta_0_err = params["delta_0_err"]
alpha_err = params["alpha_err"]
beta_err = params["beta_err"]
r0_err = params["r0_err"]

a_err = params["a_err"]
b_err = params["b_err"]

# -----
# Monte Carlo sampling
# -----

delta0_samples = np.random.normal(
    delta_0,
    delta_0_err,
    n_samples
)

alpha_samples = np.random.normal(
    alpha,
    alpha_err,
    n_samples
)

beta_samples = np.random.normal(
    beta,
    beta_err,
    n_samples
)

r0_samples = np.random.normal(
    r0,
    r0_err,
    n_samples
)

a_samples = np.random.normal(
    a,
    a_err,
    n_samples
)

b_samples = np.random.normal(
    b,
    b_err,
    n_samples
)

# -----
# Store Monte Carlo realizations
# -----

all_delta = []
all_logT = []
all_NHI = []
all_EW = []

# -----
# Evaluate the model
# -----

for (
    d0,
    al,
    be,
    r0_s,
    a_s,
    b_s
) in Zip(
    delta0_samples,
    alpha_samples,
    beta_samples,
    r0_samples,
    a_samples,
    b_samples
):
    # Reject unphysical characteristic scales.
    if r0_s <= 0:
        continue

    # Filament overdensity profile.
    delta_r = (
        1
        + d0
        / (
            1
            + (r / r0_s) ** al
        )
        ** be
    )

    # Temperature-density relation.
    log_T = (
        a_s
        * np.log10(
            delta_r
            * rho_avg
            / rho_crit
        )
        + b_s
        - 1
    )

    T = 10 ** log_T
    T4 = T / 1e4

    # Neutral hydrogen column density.
    NHI = (

```

```

        2.7e13
        * delta_r ** 1.5
        * T4 ** (-0.26)
        * Gamma_12 ** (-1)
        * ((1 + z) / 4) ** (9 / 2)
    )

    # Ly-alpha equivalent width.
    EW = 5.434e-15 * NHI

    # Store results.
    all_delta.append(delta_r)
    all_logT.append(log_T)
    all_NHI.append(NHI)
    all_EW.append(EW)

# Convert to arrays.
all_delta = np.asarray(all_delta)
all_logT = np.asarray(all_logT)
all_NHI = np.asarray(all_NHI)
all_EW = np.asarray(all_EW)

# Calculate summary statistics.
stats = {
    "delta": get_stats(all_delta),
    "logT": get_stats(all_logT),
    "NHI": get_stats(all_NHI),
    "EW": get_stats(all_EW)
}

return stats

# =====
# Model parameters
# =====

# Short-filament model
short_params = {
    "delta_0": 38.58,
    "alpha": 1.57,
    "beta": 1.0,
    "r0": 0.56,

    "delta_0_err": 0.89,
    "alpha_err": 0.12,
    "beta_err": 0.12,
    "r0_err": 0.05,

    "rho_avg": 2.4e-7,
    "rho_crit": 5.3e-6,

    "a": 0.7,
    "b": 6.09,
    "a_err": 0.03,
    "b_err": 0.01,

    "Gamma_12": 0.075,
    "z": 0.15
}

# Long-filament model
long_params = {
    "delta_0": 18.34,
    "alpha": 1.53,
    "beta": 1.0,
    "r0": 0.51,

    "delta_0_err": 1.28,
    "alpha_err": 0.32,
    "beta_err": 0.17,
    "r0_err": 0.11,

    "rho_avg": 2.4e-7,
    "rho_crit": 5.3e-6,

    "a": 0.7,
    "b": 6.09,
    "a_err": 0.03,
    "b_err": 0.01,

    "Gamma_12": 0.075,
    "z": 0.15
}

# Modified model
mod_params = {
    "delta_0": 38.58,
    "alpha": 1.57,
    "beta": 1.0,
    "r0": 3.6,

    "delta_0_err": 0.89,
    "alpha_err": 0.12,
    "beta_err": 0.12,
    "r0_err": 0.05,

    "rho_avg": 2.4e-7,
    "rho_crit": 5.3e-6,

    "a": 0.7,
    "b": 6.09,
    "a_err": 0.03,
    "b_err": 0.01,

    "Gamma_12": 0.075,
    "z": 0.15
}

# =====
# Distance grid
# =====

r = np.logspace(
    -1.5,
    2,
    200
)

```

```

# =====
# Run the three models
# =====

stats_short = run_filament_model(
    r,
    short_params,
    n_samples=5000,
    seed=42
)

stats_long = run_filament_model(
    r,
    long_params,
    n_samples=5000,
    seed=42
)

stats_mod = run_filament_model(
    r,
    mod_params,
    n_samples=5000,
    seed=42
)

# Extract EW profiles.
EW_med_short, EW_lo_short, EW_hi_short = (
    stats_short["EW"]
)

EW_med_long, EW_lo_long, EW_hi_long = (
    stats_long["EW"]
)

EW_med_mod, EW_lo_mod, EW_hi_mod = (
    stats_mod["EW"]
)

# =====
# Plot
# =====

plt.rcParams.update({
    "font.family": "DejaVu Sans",
    "font.size": 14,
    "axes.labelsize": 18,
    "axes.titlesize": 18,
    "xtick.labelsize": 14,
    "ytick.labelsize": 14,
    "legend.fontsize": 13,
    "axes.linewidth": 1.6
})

fig, ax = plt.subplots(
    figsize=(8.6, 6.8),
    dpi=300
)

# =====
# Observed data
# =====

ax.errorbar(
    x_det,
    W_det,
    fmt="o",
    color="b",
    markeredgecolor="k",
    markersize=8,
    alpha=0.8,
    label="Detection"
)

ax.errorbar(
    x_ul,
    W_ul,
    fmt="v",
    markersize=8,
    markeredgewidth=1.5,
    markeredgecolor="blue",
    color="blue",
    markerfacecolor="none",
    uplims=False,
    alpha=0.8,
    label="Upper limits"
)

# =====
# Short-filament model
# =====

ax.fill_between(
    r,
    EW_lo_short,
    EW_hi_short,
    color="#7b2cbf",
    alpha=0.18,
    zorder=2
)

ax.plot(
    r,
    EW_med_short,
    color="#5a189a",
    linewidth=2.8,
    linestyle="--",
    label="Model: Short filament",
    zorder=3
)

# =====
# Long-filament model
# =====

ax.fill_between(
    r,
    EW_lo_long,

```

```

        EW_hi_long,
        color="#2ca25f",
        alpha=0.16,
        zorder=2
    )

ax.plot(
    r,
    EW_med_long,
    color="#1b7f3a",
    linewidth=2.8,
    linestyle="-.",
    label="Model: Long filament",
    zorder=3
)

# =====
# Modified model
# =====

ax.plot(
    r,
    EW_med_mod,
    color="purple",
    linewidth=5,
    alpha=0.7,
    zorder=20
)

# =====
# Posterior EW model
# =====

ax.fill_between(
    x_grid,
    EW_0p5,
    EW_99p5,
    color="dodgerblue",
    alpha=0.3,
    zorder=10
)

sigma_int_50 = np.median(
    sigma
)

EW_low = (
    EW_50
    * 10 ** (-sigma_int_50)
)

EW_high = (
    EW_50
    * 10 ** sigma_int_50
)

ax.fill_between(
    x_grid,
    EW_low,
    EW_high,
    color="cyan",
    alpha=0.2
)

ax.plot(
    x_grid,
    EW_50,
    color="black",
    linewidth=3.2,
    label="Best-fit"
)

# =====
# Axes
# =====

ax.set_xscale("log")
ax.set_yscale("log")

ax.set_xlim(
    0.05,
    30
)

ax.set_ylim(
    3e-3,
    4
)

ax.set_xlabel(
    r"$D_{\rm Fil}$ (pMpc)",
    fontsize=22
)

ax.set_ylabel(
    r"$W_r({\rm Ly})\alpha\sim({\rm \AA})$",
    fontsize=22
)

ax.tick_params(
    axis="both",
    length=4,
    width=3,
    which="minor",
    direction="out",
    labelsiz=20,
    color="k"
)

ax.tick_params(
    axis="both",
    length=8,
    width=4,
    which="major",
    direction="out",
    labelsiz=20,
    color="k"
)

ax.yaxis.set_ticks_position("both")

```

```

ax.xaxis.set_ticks_position("both")

ax.minorticks_on()

# =====
# Legend
# =====

handles, labels = (
    ax.get_legend_handles_labels()
)

order = [
    "Detection",
    "Upper limits",
    "Model: Short filament",
    "Model: Long filament",
    "Best-fit"
]

new_handles = []
new_labels = []

for name in order:
    if name in labels:
        index = labels.index(name)

        new_handles.append(
            handles[index]
        )

        new_labels.append(
            labels[index]
        )

ax.legend(
    new_handles,
    new_labels,
    loc="upper center",
    bbox_to_anchor=(0.5, 1.2),
    ncol=2,
    fontsize=16,
    frameon=True,
    fancybox=True,
    framealpha=0.95,
    edgecolor="0.85",
    columnspacing=1.2,
    handlelength=2.5
)

plt.tight_layout()
plt.show()

```

```

"""
filament_baryon_fraction.py

Calculate the neutral-hydrogen mass, filament hydrogen density,
baryon density parameter, and baryon fraction associated with
the detected cosmic-web filaments.

The calculation proceeds by:

1. Integrating the model Ly-alpha EW profile to obtain the
   total neutral-hydrogen mass.
2. Converting the mass into a filament H I density.
3. Using the adopted neutral-hydrogen fraction (f_HI) to
   infer the total hydrogen density.
4. Converting the hydrogen density into cosmological density
   parameters and the baryon fraction relative to the cosmic
   baryon density.

Inputs
-----
Filament_sample_318.csv

Calculated quantities
-----
rho_HI : neutral-hydrogen density of the filament
        [Msun Mpc^-3]

User inputs
-----
f_HI : neutral-hydrogen fraction

Outputs
-----
Neutral-hydrogen mass
Filament H I density
Hydrogen number density
Hydrogen overdensity
Omega_HI
Omega_H
Omega_b
Omega_b / Omega_b(z)
"""

# =====
# Imports
# =====

import numpy as np
import pandas as pd
from scipy import integrate

# =====
# Input data
# =====

INPUT_FILE = "Filament_sample_318.csv"

data = pd.read_csv(INPUT_FILE)
rb_data_all = data.copy()

# Flag used for the EW analysis.
rb_data_all["det_flag"] = (
    rb_data_all["EW_total"] != -9999
).astype(int)

# Use EW_noise for non-detections and measured EW for
# detections.
rb_data_all["ew"] = np.where(
    rb_data_all["flag"] == 0,
    rb_data_all["ew_noise_lya"],
    rb_data_all["EW_total"]
)

df = rb_data_all.copy()

# =====
# Cosmological parameters
# =====

z = 0.15

H0 = 67.4 # km s^-1 Mpc^-1
Omega_m0 = 0.315 # Present-day matter density parameter
Omega_b0 = 0.0493 # Present-day baryon density parameter
Omega_lambda0 = 0.685 # Present-day dark-energy density parameter

#G = 4.302e-9 # Mpc (km/s)^2 Msun^-1

# =====
# EW model
# =====

alpha = 0.07 # Best fit value for our EW profile
beta = -0.27

D_fill = 3.6 # Characteristic length of filament in Mpc
z0 = 8682.0 # Total filament length in Mpc

# Conversion constants
m_p = 8.4089e-58 # proton mass in Msun
Mpc_to_cm = 3.086e24 # cm

# EW-N_HI conversion factor
A = 5.43e-15

def EW(r):
    """
    Ly-alpha equivalent-width profile.

    Parameters
    -----
    r : float or ndarray
        Filament-centric distance in Mpc.

    Returns
    -----
    float or ndarray
        Equivalent width in Angstrom.
    """

```

```

    """
    return 10 ** (
        -alpha * (r / D_fill) + beta
    )

def N_HI(r):
    """
    Neutral-hydrogen column-density profile.

    Parameters
    -----
    r : float or ndarray
        Filament-centric distance in Mpc.

    Returns
    -----
    float or ndarray
        N_HI in cm^-2.
    """

    return EW(r) / A

# =====
# H I mass calculation
# =====

R0 = D_fill

radial_integral, integration_error = integrate.quad(
    N_HI,
    -R0,
    R0
)

# Line-of-sight extent used in the mass calculation.
z_factor = z0

result = (
    radial_integral
    * z_factor
    * Mpc_to_cm**2
)

M_HI = m_p * result

print("=" * 55)
print("INTEGRATION RESULTS FOR MASS")
print("=" * 55)

print(
    f"M_HI = {M_HI:.3e} Msun"
)

# =====
# Filament H I density
# =====

# Number of cluster fields and adopted volume per field.
N_FIELDS = 318
VOLUME_PER_FIELD = 40.0**3      # Mpc^3

VOLUME_FILAMENT = (
    N_FIELDS * VOLUME_PER_FIELD
)

density = (
    M_HI / VOLUME_FILAMENT
)

print("=" * 55)
print("DENSITY OF THE FILAMENT")
print("=" * 55)

print(
    f"Filament volume = "
    f"{VOLUME_FILAMENT:.3e} Mpc^3"
)

print(
    f"H I density = "
    f"{density:.3e} Msun Mpc^-3"
)

# =====
# Cosmological baryon density
# =====

Ez2 = (
    Omega_m0 * (1.0 + z) ** 3
    + Omega_lambda0
)

Omega_b_z = (
    Omega_b0
    * (1.0 + z) ** 3
    / Ez2
)

print(
    f"Omega_b(z={z}) = "
    f"{Omega_b_z:.4f}"
)

# =====
# Density to hydrogen overdensity
# =====

def density_to_overdensity(
    rho_msun_mpc3,
    z=0.15
):
    """
    Convert hydrogen mass density to hydrogen number density
    and baryon overdensity relative to the cosmic mean
    hydrogen density.

```

```

Parameters
-----
rho_msun_mpc3 : float
    Hydrogen mass density in Msun Mpc^-3.

z : float
    Redshift.

Returns
-----
nH : float
    Hydrogen number density in cm^-3.

nH_mean : float
    Mean cosmic hydrogen density in cm^-3.

overdensity : float
    Hydrogen overdensity, 1 + delta.
"""

MSUN = 1.98847e33      # g
MPC = 3.08567758e24    # cm
MP = 1.6726219e-24     # g

# Convert mass density to cgs.
rho_g_cm3 = (
    rho_msun_mpc3
    * MSUN
    / MPC**3
)

# Hydrogen number density.
nH = rho_g_cm3 / MP

# Mean cosmic hydrogen density.
nH_mean = (
    1.1e-5
    * ((1.0 + z) / 4.0) ** 3
)

# Hydrogen overdensity.
overdensity = (
    nH / nH_mean
)

return (
    nH,
    nH_mean,
    overdensity
)

# =====
# Filament physical properties
# =====

rho_HI = density

f_HI = float(
    input("Please enter f_HI: ")
)

n_H, n_H_mean, overdensity_H = density_to_overdensity(
    rho_HI,
    z=0.15
)

# =====
# Cosmological density parameters
# =====
# Adopted critical density for the filament baryon calculation
rho_crit = 1.5e11      # Msun Mpc^-3

Omega_HI = (
    rho_HI / rho_crit
)

Omega_H = (
    Omega_HI / f_HI
)

# Include helium contribution.
Omega_b_Fil = (
    1.245 * Omega_H
)

f_b_Fil = (
    Omega_b_Fil / Omega_b_z
)

# =====
# Results
# =====

print("=" * 55)
print("FILAMENT PHYSICAL PROPERTIES")
print("=" * 55)

print(
    f"rho_HI = "
    f"{rho_HI:.3e} Msun Mpc^-3"
)

print(
    f"n_H = "
    f"{n_H:.3e} cm^-3"
)

print(
    f"Mean n_H = "
    f"{n_H_mean:.3e} cm^-3"
)

print(
    f"1 + delta_H = "
    f"{overdensity_H:.2f}"
)

print(
    f"log10(n_H / cm^-3) = "

```

```
        f"{np.log10(n_H):.2f}"
    )

print("=" * 55)
print("COSMOLOGICAL QUANTITIES")
print("=" * 55)

print(
    f"Omega_HI, Fil      = "
    f"{Omega_HI:.2e}"
)

print(
    f"Omega_H, Fil      = "
    f"{Omega_H:.4f}"
)

print(
    f"Omega_b, Fil      = "
    f"{Omega_b_Fil:.4f}"
)

print(
    f"f_b, Fil          = "
    f"{f_b_Fil:.2f}"
)
```

```

"""
IGM_CF.py

Calculate the expected IGM Ly-alpha covering fraction and its
model uncertainty for a specified equivalent-width threshold.

The calculation converts the equivalent-width threshold into an
approximate Ly-alpha column density, evaluates the absorber incidence
dN/dz, and converts this incidence into a covering fraction over the
adopted velocity interval.

This module is used by covering_fraction.py.
"""

import numpy as np

# =====
# Physical constants
# =====

SPEED_OF_LIGHT = 300000.0 # km/s

# =====
# Column density
# =====

def column_density(ew_th, w_lya, f):
    """
    Calculate the Ly-alpha column density corresponding to
    an equivalent-width threshold.

    Parameters
    -----
    ew_th : float
        Equivalent-width threshold in Angstrom.
    w_lya : float
        Rest wavelength of the transition in Angstrom.
    f : float
        Oscillator strength of the transition.

    Returns
    -----
    float
        Column density in cm^-2.
    """
    ew_cm = ew_th * 1e-8
    wavelength_cm = w_lya * 1e-8

    return (
        ew_cm
        / (8.85e-13 * f * wavelength_cm**2)
    )

# =====
# Absorber incidence
# =====

def dNdz(N, z):
    """
    Calculate the number of absorbers per unit redshift.

    The adopted redshift-dependent absorber distribution is

        dN/dz = C14 * (N / 1e14)^(1 - b_z),

    where

        C14 = 16 * (1 + z)^gamma
        b_z = 1.75 - 0.31*z
        gamma = 2.3.

    Parameters
    -----
    N : float
        Column density in cm^-2.
    z : float
        Redshift.

    Returns
    -----
    float
        Absorber incidence per unit redshift.
    """
    gamma = 2.3

    C14 = 16.0 * (1.0 + z)**gamma
    b_z = 1.75 - 0.31 * z

    return C14 * (N / 1e14)**(1.0 - b_z)

# =====
# Model uncertainty
# =====

def model_uncertainty_from_b(N, z):
    """
    Calculate the uncertainty in dN/dz arising from the
    uncertainty in the absorber-distribution slope.

    Parameters
    -----
    N : float
        Column density in cm^-2.
    z : float
        Redshift.

    Returns
    -----
    float
        Uncertainty in dN/dz.
    """
    R = dNdz(N, z)

    # Adopted uncertainty in b_z.
    sigma_bz = np.sqrt(
        0.03**2 + (0.10 * z)**2
    )

```

```

    )

    # Propagation of uncertainty through dN/dz.
    sigma_R = (
        abs(np.log(N / 1e14))
        * R
        * sigma_bz
    )

    return sigma_R

# =====
# IGM covering fraction
# =====

def IGM_CF(ew_th, w_lya, f, v, z):
    """
    Calculate the expected IGM covering fraction and its
    model uncertainty.

    Parameters
    -----
    ew_th : float
        Equivalent-width threshold in Angstrom.
    w_lya : float
        Rest wavelength of the transition in Angstrom.
    f : float
        Oscillator strength of the transition.
    v : float
        Velocity interval in km/s.
    z : float
        Redshift.

    Returns
    -----
    CF : float
        Expected IGM covering fraction.
    CF_err : float
        Model uncertainty in the covering fraction.
    """

    # Convert EW threshold to column density.
    N = column_density(
        ew_th,
        w_lya,
        f
    )

    # Absorber incidence per unit redshift.
    incidence = dNdz(
        N,
        z
    )

    # Uncertainty in absorber incidence.
    incidence_err = model_uncertainty_from_b(
        N,
        z
    )

    # Expected number of absorbers in the velocity interval.
    exponent = (
        -incidence
        * v
        * (1.0 + z)
        / SPEED_OF_LIGHT
    )

    # Poisson probability of at least one absorber.
    CF = 1.0 - np.exp(exponent)

    # Propagate uncertainty from dN/dz.
    derivative = (
        v
        * (1.0 + z)
        / SPEED_OF_LIGHT
        * np.exp(exponent)
    )

    CF_err = derivative * incidence_err

    return CF, CF_err

```